Semester 01 2025/2026

Subject	**:**	**Database Programming (SECP3623)**
Section	**:**	**Section 1-2**
Task	**:**	**Project II**
Due	**:**	**16th January 2026 (10%)**

Project Overview

This mini project enables students to design and implement a **NoSQL database backend** using **MongoDB**. Students are expected to apply the concepts learned in the NoSQL and MongoDB topics by modelling a selected domain, storing data in collections, and performing CRUD operations, indexing, and aggregation workflows.

Note: *This project focuses on* ***MongoDB backend development only****.*
No front-end or UI is required.

Through this project, students will:
- Apply NoSQL principles and the **Document Data Model**.
- Demonstrate CRUD operations in MongoDB (insert, find, update, delete).
- Use MongoDB **query operators, projections, and update operators**.
- Implement **indexing** to improve performance.
- Build **aggregation pipelines** for analytics.
- Analyse NoSQL security considerations and design challenges.
- Provide output samples through a structured report and presentation.

By the end of this project, each group (3–4 members) should be able to demonstrate a working NoSQL backend capable of supporting common non-relational workloads such as activity management, content systems, event systems, logging systems, or any domain of choice.

Project Requirements

A. NoSQL Data Modelling
Each group must:
1. Select a real-world domain
   (examples: event management, booking system, delivery records, learning portal, hobby communities).
2. Design **at least 3–4 MongoDB collections** using Document data model.
3. Provide sample document structures showing:
   - Embedded documents
   - References (if applicable)
   - Arrays
   - Appropriate data types (String, Number, Date, Array, Object)

B. Basic MongoDB Operations (CRUD)
Students must implement:

1. **Insert Operations**
   - insertOne()
   - insertMany()
2. **Query Operations**
   - find(), findOne()
   - Condition filters using $gt, $lt, $eq, $in, $and, $or
   - Array queries
   - Projection (include/exclude fields)
3. **Update Operations**
   - updateOne(), updateMany()
   - Update operators:
     - $set, $inc, $push, $pull
4. **Delete Operations**
   - deleteOne(), deleteMany()

C. Advanced MongoDB Operations

1. Indexing
Create at least **two indexes**, including:

- Single-field index
- Compound index

Explain why the selected fields were indexed.

2. Aggregation Pipelines
Use at least **two pipeline stages**, for example:

- $match
- $group
- $project
- $sort
- $limit

Examples of acceptable aggregation outputs:

- Total counts
- Group by category
- Top N items
- Summaries (averages, totals)

3. Sorting
Demonstrate sorting records in ascending and descending order.

4. Query Performance Discussion
Explain how indexes influence query speed
(no benchmarking required; conceptual explanation is enough).

5. Security & Challenges
Briefly discuss:

- Security concerns in NoSQL systems
- Access control, injection risks
- Consistency trade-offs in distributed systems

Project Report Requirements

Your report must follow this structure:

1. Introduction
    - Brief description of the chosen domain
    - Project objectives
    - Team members & roles
2. NoSQL Data Model
    - List and explanation of collections
    - Example JSON documents
    - Explanation of embedding vs referencing
    - Data types used
3. CRUD Implementation Summary
    - Explanation of insert, query, update, delete operations
    - Screenshots of MongoDB Compass or shell execution
    - Projections and query tests
4. Advanced Operations
    - Index creation examples
    - Aggregation pipeline examples
    - Sorting demonstrations
    - Explanation of performance improvements
    - Any challenges encountered
5. Security & Limitations
    - Basic access controls
    - Injection risk
    - Database constraints or limitations
6. Teamwork
    - Each member describes role & contributions
7. Conclusion
    - What was learned
    - Challenges
    - Recommendations for improvement

Tips for report writing:

- Use screenshots with labels and captions.
- Do not paste all codes include only representative examples.
- Ensure consistent formatting and clear headings.

Presentation Requirements

- **Duration:** 10 minutes per group.
- **Tools:** MySQL Workbench + PowerPoint.
- **Presentation flow:**
    1. Brief case study introduction
    2. NOSQL modelling explanation
    3. CRUD demonstrations
    4. Indexing and Aggregation
    5. Team Member contributions

Submission Items

1. Project Report + Presentation URL
2. Presentation Slides
3. Folder of .json / .js MongoDB commands
4. Exported collections (optional)

Marking Rubric

NOSQL Implementation (CLO2)

Criteria	Excellent (5)	Good (4)	Fair (3)	Weak (2)	Poor (1)	Marks
NoSQL Modelling (Collections, Documents, Data Types)	Well-designed collections with correct embedding/referencing; strong justification; consistent and valid data types.	Good data model with minor inconsistencies; generally correct structure.	Basic model; limited justification; inconsistent data organisation.	Weak or incorrect model; unclear structure; poor justification.	Not attempted or completely incorrect modelling.	
CRUD Operations & Query Logic	CRUD fully functional; effective use of filters, projections, and operators; commands execute flawlessly.	CRUD mostly correct; minor errors in operators or projections.	CRUD partially correct; limited operator use; inconsistent outputs.	Minimal working CRUD; poor filtering logic.	No meaningful CRUD attempted.	
Indexing & Aggregation Pipelines	Correct index creation (single & compound); meaningful aggregation pipeline; clear explanation of outcomes.	Indexes implemented; aggregation attempted with minor issues.	Partial indexing or basic aggregation with limited insight.	Minimal attempt; unclear or incorrect results.	Not attempted.	
Interpretation & Justification	Strong explanation of how NoSQL design, queries, indexing, and aggregation support system needs; clear reasoning.	Good explanation with minor gaps.	Basic interpretation with little linkage to system logic.	Minimal explanation; unclear or incorrect reasoning.	Not attempted or irrelevant explanation.	
Subtotal						

Teamwork and Collaboration (CLO3)

Criteria	Excellent (5)	Good (4)	Fair (3)	Weak (2)	Poor (1)	Marks
Team Coordination & Contribution	Clear task division; balanced participation; strong collaboration evident in report and presentation; each member contributes to SQL work.	Good teamwork and participation with minor imbalance.	Some collaboration; limited task clarity.	Minimal teamwork; dominated by few members.	No teamwork evidence; incomplete group submission.	
Subtotal						

Documentation & Presentation (CLO4)

Criteria	Excellent (5)	Good (4)	Fair (3)	Weak (2)	Poor (1)	Marks
Report Documentation	Clear, complete report including introduction, NoSQL modelling, CRUD screenshots, indexing, aggregation, and reflection.	Report complete with minor formatting issues.	Adequate report but lacks detail, structure, or screenshots.	Weakly structured or several missing sections.	Missing or very poor documentation.	
Slide Quality & Visual Communication	Slides are professional, organised, visually clear, and consistent with flow.	Good slides with minor design issues.	Understandable slides but inconsistent layout.	Poorly structured or cluttered slides.	Missing or unreadable slides.	
Verbal Presentation	Excellent delivery, confident speakers, balanced time use, fluent explanation, strong Q&A.	Good delivery with minor coordination issues.	Understandable but low energy or partial clarity.	Disorganized or weak explanations.	Unable to present or answer questions.	
Subtotal						